

SQLP 과목 III 튜닝 스타일 모의문제 · 1회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

Sequential 액세스와 Random 액세스, 그리고 그에 동반되는 블록 단위 I/O에 대한 설명으로 가장 적절하지 않은 것은?

- ① db file sequential read 대기 이벤트는 이름과 달리 한 번에 한 블록씩 읽어 들이는 Single Block I/O 도중에 발생하며, 주로 인덱스 기반 Random 액세스에서 나타난다.
- ② Full Table Scan은 세그먼트에 속한 익스텐트 내 블록들을 물리적으로 저장된 순서대로 훑어 나가는 Sequential 액세스이며, 이때 Multiblock I/O가 동반된다.
- ③ 인덱스를 경유해 테이블을 액세스할 때는 ROWID가 가리키는 블록들이 대체로 인접해 있어, 한 번의 I/O Call로 여러 블록을 함께 읽어 들이는 Multiblock I/O가 사용된다.
- ④ 인덱스 리프 블록을 논리적 연결 순서에 따라 좌에서 우로 스캔하는 것은 Sequential 액세스이며, 스캔 범위가 넓을수록 읽는 리프 블록 수가 늘어난다.

2

주문상세 테이블(약 3,000만 건)에서 특정 배송단계에 걸린 주문 12,600건의 상태코드를 일괄 변경하는 UPDATE의 트레이스이다. 이 SQL 한 번의 수행에 대한 버퍼 캐시 히트율(BCHR)로 가장 적절한 것은?

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.011	0.013	0	0	0	0
Execute	1	6.320	22.480	50000	920000	80000	12600
Fetch	0	0.000	0.000	0	0	0	0
Total	2	6.331	22.493	50000	920000	80000	12600

- ① 약 94.6%
- ② 약 95.2%
- ③ 약 90.5%
- ④ 약 95.0%

3

쿼리를 실제로 수행하지 않고 옵티마이저가 세운 예상(추정) 실행계획만 확인하려 한다. 각 DBMS의 도구 사용에 대한 설명으로 가장 적절한 것은?

- ① 오라클 — SET SHOWPLAN_TEXT ON 을 실행한 뒤 SELECT문을 던지면, 문장을 수행하지 않고 예상 실행계획만 텍스트로 반환한다.
- ② 오라클 — SQL*Plus에서 SET AUTOTRACE TRACEONLY EXPLAIN 설정 후 SELECT문을 실행하면, 문장을 수행하지 않고 예상 실행계획만 출력된다.
- ③ SQL Server — EXPLAIN PLAN FOR SELECT ...; 로 계획을 적재한 뒤 SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY); 로 조회하면 예상 실행계획이 표시된다.
- ④ SQL Server — SET STATISTICS PROFILE ON 을 실행한 뒤 쿼리를 던지면, 쿼리를 수행하지 않고 예상 실행계획만 반환한다.

4

회원 테이블(약 1,800만 건)과 접속이력 테이블(약 240만 건)을 조인해 집계하는 SQL의 TKPROF 결과이다. 이 트레이스의 해석으로 가장 적절하지 않은 것은?

아 래

Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.003	0.003	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	121	3.147	28.600	72400	810300	0	1800
Total	123	3.150	28.603	72400	810300	0	1800
Rows Row Source Operation							
1800	HASH JOIN (cr=810300 pr=72400 pw=0 time=28551000 us)						
2400000	TABLE ACCESS FULL 접속이력 (cr=72300 pr=12400 pw=0 time=1802000 us)						
18000000	TABLE ACCESS FULL 회원 (cr=738000 pr=60000 pw=0 time=25640000 us)						

- ① Elapsed Time(28.6초)과 CPU Time(3.15초)의 약 25.5초 간극은 대기 시간이며, 72,400개 디스크 블록의 물리 I/O 대기가 가장 유력한 원인이다.
- ② 이 실행의 BCHR은 약 91.1%로, 논리적 I/O 약 81만 블록 중 약 9%를 디스크에서 물리적으로 읽었다.
- ③ 두 Full Scan 중 회원 테이블(cr=738,000)이 전체 블록 I/O의 약 91%를 차지해 블록 소비의 주된 지점이다.
- ④ CPU Time이 3.15초로 집계됐으므로 이 SQL의 주된 병목은 CPU 연산이고, 인스턴스 CPU 자원 증설이 최우선 처방이다.

5

배송내역 테이블(약 620만 건)에는 결합 인덱스 배송_IX = (지점코드, 배송일자)가 있다. 지점명을 함께 얻기 위해 지점 테이블(기본키 지점_PK = 지점코드)과 조인하는 다음 SQL의 실행계획이다.

```
SELECT b.배송번호, b.배송일자, b.배송상태, j.지점명
FROM 배송내역 b, 지점 j
WHERE b.지점코드 = 'S12'
AND b.배송일자 >= DATE '2026-06-01'
AND b.배송일자 < DATE '2026-07-01'
AND b.배송상태 = '완료'
AND j.지점코드 = b.지점코드;
```

이 실행계획에 대한 설명으로 가장 적절한 것은?

아 래

Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(285	1850	96200)
1	0	NESTED LOOPS	(285	1850	96200)
2	1	TABLE ACCESS BY INDEX ROWID 배송내역	(280	1850	64750)
3	2	INDEX RANGE SCAN 배송_IX	(18	7400	)
4	1	TABLE ACCESS BY INDEX ROWID 지점	(1	1	17)
5	4	INDEX UNIQUE SCAN 지점_PK	(0	1	)

- ① 지점코드와 배송일자는 배송_IX를 통한 인덱스 액세스 조건으로 처리되고, 인덱스에 없는 배송상태는 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 걸러진다.
- ② 배송상태 조건도 배송_IX의 인덱스 액세스 조건에 포함되어 INDEX RANGE SCAN의 스캔 범위를 좁히는 데 사용된다.
- ③ 선두 칼럼 지점코드에 SUBSTR(지점코드,1,2)='S1'처럼 가공을 가하더라도 배송_IX의 INDEX RANGE SCAN은 그대로 유지된다.
- ④ INDEX RANGE SCAN이 반환한 7,400건이 곧 이 SQL의 최종 결과 건수이며, TABLE ACCESS BY INDEX ROWID 단계에서는 건수 감소가 일어나지 않는다.

6

여러 컬럼으로 구성된 결합 인덱스의 컬럼 순서와 선두 컬럼 조건에 대한 설명으로 가장 적절하지 않은 것은?

- ① 결합 인덱스의 선두 컬럼이 조건절에서 등치(=) 조건으로 제공되면 스캔 시작점과 끝점이 좁게 확정되어, 실제로 읽어야 할 인덱스 범위가 줄어든다.
- ② 선두 컬럼이 조건절에서 누락되면, Index Skip Scan 같은 예외적 경로를 제외하고는 그 인덱스를 정상적인 Range Scan으로 활용하기 어렵다.
- ③ 등치(=) 조건 컬럼과 범위(BETWEEN, > 등) 조건 컬럼을 함께 인덱스에 구성할 때는, 범위 조건 컬럼을 앞에 두고 등치 조건 컬럼을 뒤에 두어야 인덱스 스캔 범위가 최소화된다.
- ④ 결합 인덱스의 컬럼 순서는 각 컬럼이 어떤 연산자로 조건에 쓰이는지에 따라 스캔 효율이 달라지므로, 컬럼의 저장 크기가 아니라 조건 사용 패턴을 기준으로 결정한다.

7

계약 테이블(약 480만 건)에서 계약_IX 인덱스를 경유해 조회한 SQL의 트레이스이다. 아래에 대한 설명으로 가장 적절한 것은?

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.008	0.009	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	33	8.240	41.700	38200	479500	0	480
Total	35	8.248	41.709	38200	479500	0	480
Rows	Row Source Operation						
480	TABLE ACCESS BY INDEX ROWID 계약 (cr=479500 pr=38200 pw=0 time=41180500 us)						
479812	INDEX RANGE SCAN 계약_IX (cr=642 pr=210 pw=0 time=2103400 us)						

- ① 계약_IX 인덱스의 클러스터링 팩터는 매우 좋은 상태다.
- ② 인덱스 스캔 단계(cr=642)가 이 SQL의 주된 병목이므로 인덱스를 재생성해야 한다.
- ③ CPU Time과 Elapsed Time 간 약 33초 차이는 38,200개 디스크 블록을 읽기 위한 I/O Call 대기 때문일 가능성이 가장 높다.
- ④ Fetch를 33회에 나눠 수행했기 때문에 Elapsed Time이 길어진 것이다.

8

인덱스 설계의 기준에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인덱스 컬럼의 선택도가 높을수록(같은 값이 적어 조건을 만족하는 행이 소수일수록) 인덱스를 통한 액세스 효율이 좋아진다.
- ② 조회가 빈번한 테이블일수록 인덱스를 넉넉히 추가해 두면 각 쿼리가 최적 인덱스를 고를 수 있으므로, 인덱스 개수는 조회 성능과 대체로 비례한다.
- ③ 여러 쿼리에서 반복적으로 함께 사용되는 조건 컬럼들은 각각 개별 인덱스로 만들기보다 하나의 결합 인덱스로 묶는 편이 유리한 경우가 많다.
- ④ 인덱스는 조회 성능을 높여 주는 대신 INSERT·UPDATE·DELETE 시 인덱스도 함께 갱신되는 부하를 유발하므로, DML이 잦은 테이블에서는 인덱스 개수를 신중히 관리해야 한다.

9

NL(Nested Loops) 조인의 드라이빙 테이블 선택과 액세스 특성에 대한 설명으로 가장 적절하지 않은 것은?

- ① inner 테이블은 드라이빙 테이블에서 넘어온 값으로 반복 탐색되므로, 조인 컬럼에 인덱스가 있으면 Random 액세스로 효율적으로 처리된다.
- ② NL 조인에서 드라이빙(outer) 테이블로는 조인에 참여하는 행이 많은 큰 집합을 고를수록 inner 테이블 반복 액세스 횟수가 줄어 유리하다.
- ③ inner 테이블의 조인 컬럼에 적절한 인덱스가 없으면 드라이빙 건마다 테이블을 훑게 되어, 반복 횟수가 많을수록 성능이 급격히 나빠진다.
- ④ NL 조인은 드라이빙 테이블에서 행을 읽는 순서대로 결과를 만들어 나가므로, 앞쪽 일부 결과만 먼저 필요한 부분범위처리에 유리하다.

10

두 테이블을 해시 조인(Hash Join)으로 연결한다. 해시 조인의 수행 원리와 Build Input에 대한 설명으로 가장 적절한 것은?

- ① 해시 조인은 등치(=) 조건뿐 아니라 부등호(BETWEEN, >) 조인 조건에도 두루 활용할 수 있다.
- ② Build Input이 Hash Area에 다 담기지 못하면 조인 자체가 수행되지 못하고 오류로 종료된다.
- ③ 해시 조인은 Build Input을 다 읽기 전에 첫 결과 행을 반환할 수 있어, 부분범위 처리(빠른 첫 행 응답)에 특히 적합하다.
- ④ 두 입력 중 크기가 작을 것으로 추정되는 쪽을 Build Input으로 선택하면, 해시 테이블이 가용 메모리(Hash Area) 안에 자리 잡을 가능성이 커진다.

11

주문 테이블(약 90만 건)과 주문상세 테이블(약 540만 건)을 조인하는 SQL의 실행계획이다. 주문상세에는 인덱스 주문상세_IX = (주문번호)가 있다. 이 실행계획에 대한 설명으로 옳은 것은?

아래

Id	Operation	Name
0	SELECT STATEMENT	
1	NESTED LOOPS	
2	TABLE ACCESS FULL	주문
3	TABLE ACCESS BY INDEX ROWID	주문상세
* 4	INDEX RANGE SCAN	주문상세_IX

- ① 후행 테이블 주문상세가 드라이빙(선행) 집합이고, 주문은 각 주문상세 행마다 한 번씩 조회된다.
- ② 주문이 드라이빙(선행) 집합이며, 주문 한 건마다 주문상세_IX의 INDEX RANGE SCAN이 반복 수행되어 주문상세가 조인된다.
- ③ NESTED LOOPS 대신 두 테이블을 각각 정렬한 뒤 병합하는 방식이므로, 주문상세_IX를 경유한 테이블 랜덤 액세스는 발생하지 않는다.
- ④ 주문상세를 한 번의 TABLE ACCESS FULL로 모두 읽어 해시 영역에 적재한 뒤, 주문을 그 해시 테이블에 탐침(probe)한다.

12

비용 기반 옵티마이저(CBO)와 옵티마이저 통계정보에 대한 설명으로 가장 적절하지 않은 것은?

- ① CBO는 테이블 행 수·블록 수·컬럼 NDV·히스토그램 같은 통계정보를 바탕으로 실행계획 후보들의 예상 비용을 산정하고, 그중 최소 비용의 계획을 선택한다.
- ② 통계를 정확하게 수집할수록 옵티마이저의 계획 탐색(파싱) 작업이 짧아지므로, 통계가 최신일수록 하드파싱에 드는 시간도 함께 줄어든다.
- ③ 히스토그램은 특정 컬럼 값의 분포가 균일하지 않을 때 선택도 추정의 정밀도를 높이는 데 도움이 된다.
- ④ 통계정보가 실제 데이터 분포와 어긋나면(예: 오래되어 부정확) CBO가 카디널리티를 잘못 추정해 비효율적인 계획을 고를 수 있다.

13

세 세션이 논리적으로 같은 결과를 얻으려고 아래 SQL을 각각 수행했다. 세 SQL은 대소문자·공백·주석만 다르다. 수행 뒤 라이브러리 캐시를 요약한 결과가 함께 주어졌다. 커서 공유(Cursor Sharing)에 대한 설명으로 가장 적절한 것은?

아래

```
-- 세션 A
select * from orders where cust_id = 100;

-- 세션 B (SELECT 를 대문자로)
SELECT * from orders where cust_id = 100;

-- 세션 C (from 앞 공백 2칸 + 끝에 주석)
select * from orders where cust_id = 100; -- daily batch

[ 라이브러리 캐시 요약 ]
```

SQL_ID	SQL_TEXT (선두 요약)	PARSE_CALLS	EXECUTIONS
9k2m7b3d0x1aq	select * from orders where cust_id = 100	1	1
7f3p5c8e2y6bh	SELECT * from orders where cust_id = 100	1	1
2d5x1a4f9z0ck	select * from orders where cust_id = 100 --...	1	1

- ① 주석은 실행계획에 영향을 주지 않으므로, 주석만 다른 SQL은 항상 같은 커서를 공유한다.
- ② 오라클은 파싱 전에 SQL 텍스트를 대문자로 바꾸고 여백·주석을 제거해 정규화하므로, 세 SQL은 하나의 커서를 공유한다.
- ③ 세 SQL이 동일한 상수값(100)을 사용하므로, 바인드 변수를 쓰지 않아도 커서가 자동으로 공유된다.
- ④ 세 SQL은 대소문자·공백·주석만 다르고 의미는 같지만, 텍스트가 문자 단위로 일치하지 않아 서로 다른 SQL_ID로 각각 하드파싱되었다.

14

옵티마이저의 쿼리 변환(Query Transformation) — View Merging, 서브쿼리 Unnesting, 조건절 Pushing — 에 대한 설명으로 가장 적절한 것은?

- ① 옵티마이저가 뷰를 머지(View Merging)하면 뷰 블록과 바깥 쿼리가 하나로 합쳐져, 조인 순서·액세스 경로 선택의 폭이 넓어진다.
- ② 서브쿼리 Unnesting은 조인을 서브쿼리 형태로 되돌려, 옵티마이저가 중첩 루프(Nested Loops) 수행을 택하도록 유도하는 변환이다.
- ③ 조건절 Pushing(Predicate Pushing)은 뷰 안의 조건을 바깥 쿼리로 끌어올려, 최종 결과를 만든 직후에 필터링하는 기법이다.
- ④ GROUP BY가 포함된 복합 뷰는 머지 대상이 될 수 없어, 옵티마이저는 뷰를 개별 실행한 뒤에만 조인을 수행한다.

15

대량 이관 배치에서 아래와 같이 **append** 힌트와 **nologging** 속성을 함께 사용해 데이터를 적재한다. 이 **Direct Path Insert** 방식의 효과로 가장 적절하지 않은 것은?

아래

```
alter table sales_archive nologging;

insert /*+ append */ into sales_archive
select * from sales
where sale_dt < date '2025-01-01';

commit;
```

- ① 데이터를 버퍼 캐시를 우회해 세그먼트 HWM(고수위) 위쪽에 직접 기록하므로, 버퍼 캐시 경합이 줄어든다.
- ② 대상 테이블이 **nologging**이고 **append**가 적용되면, 삽입되는 데이터에 대한 Redo 발생이 최소화된다.
- ③ **append** 힌트는 대상 테이블에 배타적(TM) Lock을 걸지 않으므로, 다른 세션이 동시에 같은 테이블에 INSERT를 수행할 수 있다.
- ④ HWM 아래에 비어 있는 블록(삭제로 생긴 여유 공간)을 재사용하지 않고 HWM 위에 기록하므로, 세그먼트가 커질 수 있다.

16

매출 테이블은 매출일자(DATE 타입)를 기준으로 월별 RANGE 파티셔닝되어 있다(아래 DDL). 각 선택지는 **SELECT SUM(매출금액) FROM 매출 WHERE ...**의 WHERE 절이다. **Partition Pruning**이 효과적으로 작동하여 필요한 파티션만 읽는 것으로 보기 가장 적절하지 않은 것은?

아래

```
CREATE TABLE 매출 (
  매출번호    NUMBER,
  매출일자    DATE,
  매출금액    NUMBER,
  지정코드    VARCHAR2(4)
)
PARTITION BY RANGE (매출일자) (
  PARTITION p_202601 VALUES LESS THAN (DATE '2026-02-01'),
  PARTITION p_202602 VALUES LESS THAN (DATE '2026-03-01'),
  PARTITION p_202603 VALUES LESS THAN (DATE '2026-04-01'),
  PARTITION p_202604 VALUES LESS THAN (DATE '2026-05-01'),
  PARTITION p_202605 VALUES LESS THAN (DATE '2026-06-01'),
  PARTITION p_202606 VALUES LESS THAN (DATE '2026-07-01'),
  PARTITION p_max    VALUES LESS THAN (MAXVALUE)
);
```

- ① WHERE TO_CHAR(매출일자, 'YYYYMM') = '202603'
- ② WHERE 매출일자 BETWEEN DATE '2026-02-01' AND DATE '2026-02-28'
- ③ WHERE 매출일자 >= DATE '2026-05-01' AND 매출일자 < DATE '2026-07-01'
- ④ WHERE 매출일자 >= DATE '2026-03-01' AND 매출일자 < DATE '2026-04-01'

17

거래이력 테이블(약 4억 건, 8개 파티션)과 계좌 테이블(약 2,400만 건, 4개 파티션)을 병렬로 해시 조인하는 야간 배치 SQL의 실행계획이다. 이 병렬 실행계획은 두 테이블을 병렬 서버(Slave) 사이에 어떻게 분배하고 있는가? 가장 정확히 설명한 것은?

아래

Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
0	SELECT STATEMENT					
1	PX COORDINATOR					
2	PX SEND QC (RANDOM)	:TQ10002			Q1,02	P->S
3	HASH JOIN BUFFERED				Q1,02	PCWP
4	PX RECEIVE				Q1,02	PCWP
5	PX SEND HASH	:TQ10000			Q1,00	P->P
6	PX BLOCK ITERATOR		1	8	Q1,00	PCWC
7	TABLE ACCESS FULL	거래이력	1	8	Q1,00	PCWP
8	PX RECEIVE				Q1,02	PCWP
9	PX SEND HASH	:TQ10001			Q1,01	P->P
10	PX BLOCK ITERATOR		1	4	Q1,01	PCWC
11	TABLE ACCESS FULL	계좌	1	4	Q1,01	PCWP

- ① 거래이력과 계좌를 모두 조인 키의 해시 값으로 재분배(hash-hash) 하여, 각 병렬 서버가 자신에게 배정된 해시 버킷 범위의 양쪽 행만 조인한다.
- ② 거래이력은 전체를 모든 서버에 복제(broadcast)하고, 계좌는 재분배 없이 각 서버가 자기 파티션만 스캔한다(broadcast-none).
- ③ 두 테이블 모두 재분배 없이 QC(질의 조정자)로 먼저 모은 뒤, 단일 서버에서 순차적으로 조인한다.
- ④ 거래이력은 조인 키 해시로 재분배하고, 계좌는 전체를 모든 서버에 복제(broadcast)한다(hash-broadcast).

18

분석함수(윈도우 함수)와 집합 연산(UNION / UNION ALL)의 활용에 대한 설명으로 가장 적절한 것은?

- ① 분석함수(윈도우 함수)는 여러 상세 행을 그룹당 한 행으로 압축해 반환하므로, 원본 상세 행을 결과에 함께 유지할 수 없다.
- ② SUM() OVER(...) 같은 분석함수는 집계를 수행하므로 항상 GROUP BY 절과 함께 기술해야 한다.
- ③ 두 집합에 중복 행이 없음이 보장되면, UNION 대신 UNION ALL을 사용해 중복 제거를 위한 정렬(또는 해싱) 작업을 생략함으로써 비용을 줄일 수 있다.
- ④ UNION ALL의 결과 건수가 두 집합 건수의 합이라는 점에 근거하면, 중복을 제거하는 UNION의 결과 건수도 두 집합 건수의 합과 동일하다.

19

SQL Server에서 계좌 테이블을 갱신하는 두 세션(53, 57)이 동시에 수행되던 중, sys.dm_tran_locks를 조회한 결과이다. 이 락 상태에 대한 해석으로 가장 적절한 것은?

아래

resource_type	request_mode	request_status	session_id	resource_description
OBJECT	IX	GRANT	53	계좌
OBJECT	IX	GRANT	57	계좌
PAGE	IX	GRANT	53	1:8842
PAGE	IX	GRANT	57	1:8842
KEY	X	GRANT	53	(계좌번호=100-2277)
KEY	X	WAIT	57	(계좌번호=100-2277)

- ① 세션 57이 세션 53에 블로킹되어 있고, 판별 근거는 동일 KEY 자원(계좌번호=100-2277)에 대한 57의 X 요청이 WAIT이기 때문이다.
- ② 세션 53이 세션 57을 기다리고 있으며, 이 상황의 피해자(blocked)는 세션 53이다.
- ③ 세션 53과 57은 OBJECT 수준에서 IX 락을 함께 보유하므로 상호 블로킹(교착) 상태에 있다.
- ④ 세션 57의 대기는 PAGE(1:8842) 자원 경합 때문이며, 두 세션은 서로를 기다리는 교착상태(deadlock)다.

재고 테이블의 상품 'P-01' 행(초기 재고수량 = 500)을 두 트랜잭션이 아래 순서로 접근한다. t2에서 TX2가 수행한 SELECT가 반환하는 재고수량을 오라클과 SQL Server에서 순서대로 나열한 것으로 적절한 것은?
(단, 두 DBMS 모두 기본 격리수준을 조정하지 않았고, SQL Server는 READ_COMMITTED_SNAPSHOT(RCSI)을 사용하지 않는다.)

아 래		
시각	TX1	TX2
t1	UPDATE 재고 SET 재고수량 = 재고수량 - 120 WHERE 상품코드 = 'P-01'; (500 → 380, 미커밋)	
t2		SELECT 재고수량 FROM 재고 WHERE 상품코드 = 'P-01';
t3	COMMIT;	
t4		COMMIT;
초기 재고수량 = 500		

- ① 오라클 380, SQL Server 500
- ② 오라클 500, SQL Server 500
- ③ 오라클 380, SQL Server 380
- ④ 오라클 500, SQL Server 380

정답 및 해설

■ 정답 모아보기

1. ③	2. ④	3. ②	4. ④	5. ①
6. ③	7. ③	8. ②	9. ②	10. ④
11. ②	12. ②	13. ④	14. ①	15. ③
16. ①	17. ①	18. ③	19. ①	20. ④

문제 1. 정답 ③

두 액세스 방식을 그 본질부터 갈라놓고 봅시다.

Sequential 액세스: 논리적으로든 물리적으로든 서로 연결된 순서를 따라 차례로 읽어 나가는 방식입니다. 인덱스 리프 블록을 순서대로 훑는 것(논리적 순서), Full Table Scan이 익스텐트 내 블록을 순서대로 훑는 것(물리적 순서)이 여기에 해당합니다.

Random 액세스: 한 건을 얻기 위해 연결 순서와 무관한 위치를 그때그때 찾아가는 방식입니다. 인덱스에서 얻은 ROWID로 테이블의 특정 블록 하나를 집어 읽는 것이 대표적입니다.

여기에 I/O 단위가 짝지어집니다.

Sequential 액세스로 대량 블록을 훑을 때는 인접 블록을 한꺼번에 읽는 Multiblock I/O(**db file scattered read**)가 유리합니다.

Random 액세스는 흩어진 한 블록씩을 집어야 하므로 Single Block I/O(**db file sequential read**)로 처리됩니다.

③은 이 짝을 정반대로 이어 붙였습니다. 인덱스를 경유한 테이블 액세스는 ROWID가 가리키는 블록이 서로 인접하리라는 보장이 없고(그 인접 정도를 수치화한 것이 바로 클러스터링 팩터입니다), 한 건마다 블록 하나를 임의로 찾아가는 Random 액세스 + Single Block I/O입니다. "인접해서 Multiblock으로 읽는다"는 서술은 Full Scan의 성질을 인덱스 액세스에 잘못 옮겨 붙인 것입니다.

선택지	판정	이유
①	○	db file sequential read 는 이름과 달리 한 번에 한 블록을 읽는 Single Block I/O 중 발생하며, 인덱스 Random 액세스가 그 전형입니다
②	○	Full Table Scan은 익스텐트 내 블록을 물리적 순서대로 읽는 Sequential 액세스이고, 인접 블록을 묶어 읽는 Multiblock I/O가 동반됩니다
③	×	인덱스 경유 테이블 액세스는 ROWID로 블록을 임의로 찾아가는 Random 액세스·Single Block I/O입니다. 블록이 인접해 Multiblock으로 읽는다는 것은 방향을 뒤집은 서술입니다
④	○	인덱스 리프 블록을 논리적 순서로 좌에서 우로 훑는 것은 Sequential 액세스이고, 스캔 범위가 넓을수록 읽는 리프 블록 수가 증가합니다

■ 이 문제의 핵심

1. Sequential = 연결된 순서를 차례로, Random = 순서 무관한 위치를 그때그때 찾아가기. 방식이 먼저 정해지고 I/O 단위가 따라옵니다.

2. Sequential ↔ Multiblock I/O(**scattered read**), Random ↔ Single Block I/O(**sequential read**) — 대기 이벤트 이름이 직관과 반대라 헷갈리기 쉽습니다.

3. 인덱스 경유 테이블 액세스는 Random·Single Block입니다. 그 블록들이 얼마나 인접했는지를 재는 지표가 클러스터링 팩터입니다.

4. Full Table Scan은 Sequential·Multiblock으로, 넓은 범위를 대량으로 훑을 때 유리합니다.

■ 한 줄 정리 인덱스로 테이블을 읽는 것은 순서 무관한 위치를 한 블록씩 찾아가는 Random·Single Block I/O이지, 인접 블록을 한꺼번에 읽는 Multiblock I/O가 아닙니다.

문제 2. 정답 ④

BCHR은 논리적 I/O 중 디스크를 거치지 않고 버퍼 캐시에서 해결한 비율입니다. 여기서 핵심은 논리적 I/O가 Query(consistent gets) 하나가 아니라 Query + Current(db block gets)의 합이라는 점입니다. UPDATE는 읽기용

블록(Query)과 변경용 블록(Current)을 모두 만지므로 Current를 빼면 안 됩니다.

$$\text{공식 } \text{BCHR} = ((\text{Query} + \text{Current}) - \text{Disk}) / (\text{Query} + \text{Current}) \times 100$$

$$\text{논리적 I/O} = \text{Query} + \text{Current} = 920,000 + 80,000 = 1,000,000$$

$$\text{물리적 I/O} = \text{Disk} = 50,000$$

$$\begin{aligned} \text{BCHR} &= (1,000,000 - 50,000) / 1,000,000 \times 100 \\ &= 950,000 / 1,000,000 \times 100 \\ &= 95.0\% \end{aligned}$$

따라서 정답은 ④(약 95.0%)입니다.

혼동 지점을 하나씩 짚습니다.

- ① Current를 빼먹고 Query만 논리적 I/O로 봄
 $(920,000 - 50,000) / 920,000 \times 100 = 870,000 / 920,000 = 94.6\%$
 → 하위항목(Query)을 전체 논리적 I/O로 오인한 값
- ③ 분모에 Disk를 잘못 더함
 $(1,000,000 - 50,000) / (1,000,000 + 50,000) \times 100 = 950,000 / 1,050,000 = 90.5\%$
- ② Disk를 논리적 I/O에서 빼지 않고 '추가 요청'으로 분모에만 더함
 $1,000,000 / (1,000,000 + 50,000) \times 100 = 1,000,000 / 1,050,000 = 95.2\%$

세 오답 모두 분자에서 Disk를 정확히 한 번 빼고 분모는 논리적 I/O만 두는 규칙을 어긴 결과입니다.

선택지	판정	이유
①	×	Current(80,000)를 누락해 Query(920,000)만 논리적 I/O로 삼은 값입니다. 논리적 I/O = Query + Current이므로 하위항목만 쓰면 안 됩니다
②	×	Disk를 논리적 I/O에서 빼지 않고 분모에만 더한 $1,000,000/1,050,000 = 95.2\%$ 입니다. 히트한 블록 수를 잘못 잡았습니다
③	×	분모에 Disk(50,000)를 더해 $950,000/1,050,000 = 90.5\%$ 로 계산한 값입니다. Disk는 분자에서 한 번만 빼면 됩니다
④	○	논리적 I/O $1,000,000(=920,000+80,000)$ 에서 Disk 50,000을 빼 $950,000/1,000,000 = 95.0\%$ 입니다

■ 이 문제의 핵심

- 논리적 I/O = Query + Current. BCHR 계산에서 Current(db block gets)를 빠뜨리는 것이 가장 흔한 실수입니다. UPDATE·DELETE는 Current가 큼.
 - 공식은 $((\text{Query} + \text{Current}) - \text{Disk}) / (\text{Query} + \text{Current}) \times 100$. Disk는 분자에서 딱 한 번 빼고, 분모는 논리적 I/O만 둡니다.
 - Disk를 분모에 더하거나 분자에서 안 빼면 비슷하지만 다른 값(90.5%, 95.2%)이 나와 소거가 흐트러집니다.
 - BCHR이 높다고 무조건 좋은 SQL은 아닙니다 — 불필요한 논리적 I/O 자체가 많으면 히트율이 높아도 튜닝 대상입니다.
- 한 줄 정리 논리적 I/O는 Query와 Current의 합이고, BCHR은 그 합에서 Disk를 한 번 빼 다시 그 합으로 나눈 값 — 여기서는 $950,000/1,000,000 = 95.0\%$ 입니다.

문제 3. 정답 ②

먼저 두 DBMS의 도구를 소속별로 정리합니다. 이 문제의 오답은 전부 도구를 반대편 DBMS로 옮겨 붙인 것입니다.

오라클(예상 계획, 미수행)

- EXPLAIN PLAN FOR <문장> → PLAN_TABLE에 적재(화면 출력 없음)
 확인: SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
- SET AUTOTRACE TRACEONLY EXPLAIN → SELECT문을 수행하지 않고 계획만

SQL Server(예상 계획, 미수행)

- SET SHOWPLAN_TEXT ON / SET SHOWPLAN_ALL ON / SET SHOWPLAN_XML ON
 → 이후 문장을 수행하지 않고 예상 계획만 반환

SQL Server(실제 계획, 수행함)

- SET STATISTICS PROFILE ON / SET STATISTICS XML ON
 → 쿼리를 실제 수행하고 실행 행 수까지 담긴 실제 계획 반환

②은 오라클 SQL*Plus의 정통 용법입니다. SET AUTOTRACE ON 은 문장을 수행하고 계획과 통계를 함께 보여 주지만, TRACEONLY EXPLAIN 을 주면 SELECT문을 실제로 수행하지 않고 예상 실행계획만 출력합니다. "예상 계획만, 미수행"이라는 요구에 정확히 부합합니다.

나머지는 도구의 소속 DBMS가 뒤바뀌었거나(①③), 수행 여부가 반대(④)입니다.

선택지	판정	이유
①	×	SET SHOWPLAN_TEXT ON 은 SQL Server 구문입니다. 오라클에는 이 명령이 없어, 오라클에 붙이면 뒤바뀐 서술이 됩니다
②	○	오라클 SQL*Plus의 SET AUTOTRACE TRACEONLY EXPLAIN 은 SELECT문을 수행하지 않고 예상 실행계획만 출력합니다
③	×	EXPLAIN PLAN 과 DBMS_XPLAN.DISPLAY 는 오라클 도구입니다. 함수 자체는 존재하나 용도가 오라클용이라, SQL Server 절차로 제시한 것은 DBMS를 뒤바꾼 서술입니다
④	×	SET STATISTICS PROFILE ON 은 쿼리를 실제 수행해 실행치가 담긴 실제 계획을 냅니다. "수행하지 않고 예상 계획만"과 반대입니다

▪ 이 문제의 핵심

1. 예상 계획 = 미수행, 실제 계획 = 수행. 요구가 "예상"이면 문장을 돌리지 않는 도구를 골라야 합니다.
2. 오라클: **EXPLAIN PLAN + DBMS_XPLAN.DISPLAY**, 또는 **SET AUTOTRACE TRACEONLY EXPLAIN**.
3. SQL Server: 예상은 **SET SHOWPLAN_TEXT/ALL/XML ON**, 실제로는 **SET STATISTICS PROFILE/XML ON**.
4. 오답은 도구가 틀린 게 아니라 소속 DBMS가 뒤바뀐 경우가 많습니다. 명령이 어느 진영 것인지부터 확인하세요.

▪ 한 줄 정리 예상 실행계획을 미수행으로 얻는 정통 방법은 오라클의 **SET AUTOTRACE TRACEONLY EXPLAIN**(그리고 **EXPLAIN PLAN**)이며, **SHOWPLAN·STATISTICS**는 SQL Server 쪽 도구입니다.

문제 4. 정답 ④

④는 수치는 맞지만 병목의 원인을 엉뚱한 곳에 갖다 붙인 진술입니다(원인 오귀속). CPU Time이 3.15초인 것은 사실이지만, 전체 응답시간 28.6초 가운데 CPU가 차지하는 비중은 오히려 작습니다.

Elapsed - CPU = 28.600 - 3.150 = 25.45초 ← 일한 시간이 아니라 '기다란' 시간
CPU 비중 = 3.150 / 28.600 × 100 ≈ 11.0% ← 응답시간의 1/9 남짓

응답시간의 약 89%가 대기이고, 그 대기의 정체는 Row Source의 **time=** 값이 가리킵니다. 회원 테이블 Full Scan 한 노드에서만 25.64초(time=25640000 us)가 흘렀고 이 단계의 물리 읽기는 pr=60,000입니다. 즉 병목은 CPU가 아니라 회원 테이블 Full Scan의 디스크 I/O이므로, 처방은 CPU 증설이 아니라 인덱스·부분범위·집계 전략으로 Full Scan을 줄이는 것입니다. 따라서 ④가 적절하지 않습니다.

나머지 셋은 모두 옳습니다.

- ① 대기 = Elapsed - CPU = 28.600 - 3.150 ≈ 25.45초
물리 I/O Disk = 72,400 블록 → db file 읽기 대기가 유력 (참)
- ② BCHR = ((Query + Current) - Disk) / (Query + Current) × 100
= (810,300 + 0 - 72,400) / 810,300 × 100
= 737,900 / 810,300 × 100 ≈ 91.1%
→ 논리적 I/O 중 약 100 - 91.1 = 8.9% ≈ 9%를 디스크에서 읽음 (참)
- ③ 회원 cr 비중 = 738,000 / 810,300 × 100 ≈ 91.1%
→ 접속이력(72,300)보다 회원 Full Scan이 블록 소비의 주범 (참)

선택지	판정	이유
①	○	Elapsed 28.6초 - CPU 3.15초 = 약 25.5초가 대기이고, Disk 72,400 블록의 물리 I/O 대기가 그 대기의 유력한 원인입니다
②	○	(810,300 - 72,400)/810,300 = 약 91.1%로 BCHR 계산이 정확하며, 나머지 약 9%가 디스크 물리 읽기입니다
③	○	회원 cr=738,000은 전체 810,300의 약 91%로, 접속이력(72,300)보다 압도적이어서 블록 소비의 주된 지점이 맞습니다
④	×	CPU 3.15초는 응답시간의 11%뿐입니다. 병목은 회원 Full Scan의 디스크 I/O(pr=60,000, time=25.6초)이지 CPU가 아니므로 CPU 증설은 원인 오귀속입니다

▪ 이 문제의 핵심

1. CPU Time이 '측정됐다'는 사실과 'CPU가 병목이다'는 판정은 별개입니다. 비중(3.15/28.6 ≈ 11%)을 계산해야 원인이 드러납니다.
2. Elapsed - CPU가 대기 시간이고, 그 정체는 Row Source의 **pr·time=**로 특정 노드를 지목해 확인합니다.

3. 병목 노드는 **cr/pr**이 가장 큰 곳 — 여기서는 회원 Full Scan(cr=738,000, pr=60,000)입니다.
4. 옳은 처방은 병목의 원인(Full Scan I/O)을 줄이는 것이지, 병목이 아닌 자원(CPU)을 늘리는 것이 아닙니다.
 - 한 줄 정리 수치가 맞아도 병목 지목이 틀리면 오답 — CPU 3.15초는 응답시간의 11%뿐이고 진짜 병목은 회원 Full Scan의 디스크 I/O입니다.

문제 5. 정답 ①

플랜의 두 노드에서 나온 Card 값을 나란히 놓습니다.

INDEX RANGE SCAN 배송_IX Card 7,400 ← 인덱스에서 걸러진 건수
TABLE ACCESS BY INDEX ROWID 배송내역 Card 1,850 ← 테이블까지 거친 뒤 남은 건수

배송_IX는 (지점코드, 배송일자) 순서입니다. WHERE에는 **지점코드 = 'S12'**(등치)와 **배송일자** 범위가 있으므로, 선두 칼럼 등치 + 두 번째 칼럼 범위까지가 인덱스로 훑을 수 있는 연속 구간입니다. 그 결과가 INDEX RANGE SCAN의 7,400건입니다.

배송상태는 인덱스 칼럼이 아닙니다. 따라서 인덱스만으로는 걸러낼 수 없고, ROWID로 테이블 블록을 읽어 온 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 7,400건이 이 단계를 거치며 1,850건으로 줄어듭니다.

7,400 (인덱스 액세스: 지점코드=, 배송일자 범위)
— 테이블 필터(배송상태='완료') 적용 → 1,850 (최종)

즉 ①이 이 구조를 정확히 서술합니다. 배송상태를 인덱스 액세스로 오해한 ②, 선두 칼럼 가공에도 range scan이 유지된다는 ③, 인덱스 반환 건수를 최종 건수로 본 ④는 모두 이 Card 흐름과 어긋납니다.

선택지	판정	이유
①	○	지점코드(등치)·배송일자(범위)는 인덱스 액세스 조건, 배송상태는 인덱스에 없어 TABLE ACCESS 단계 필터 — Card가 7,400→1,850으로 줄어드는 흐름과 일치합니다
②	×	배송상태는 배송_IX 구성 칼럼이 아니므로 인덱스 액세스 조건이 될 수 없습니다. 그것이 스캔 범위를 좁혔다면 INDEX RANGE SCAN Card가 이미 1,850이었을 것입니다
③	×	선두 칼럼을 SUBSTR 등으로 가공하면 인덱스 정렬 순서를 탈 수 없어 INDEX RANGE SCAN 자체가 성립하지 않습니다. 경계를 뭉개진술입니다
④	×	7,400은 인덱스 단계 Card일 뿐이고, 배송상태 필터를 거친 최종 Card는 1,850입니다. 테이블 액세스 단계에서 건수 감소가 분명히 일어납니다

▪ 이 문제의 핵심

1. 결합 인덱스는 선두 칼럼 등치 + 다음 칼럼 범위까지가 액세스 조건. 여기서는 지점코드(=)·배송일자(범위)가 INDEX RANGE SCAN 구간을 만듭니다.
2. 인덱스에 없는 칼럼은 테이블 필터. 배송상태는 ROWID로 테이블을 읽은 뒤 TABLE ACCESS BY INDEX ROWID 단계에서 걸러집니다.
3. 두 노드의 Card 차이(7,400 vs 1,850)가 곧 테이블 필터의 효과입니다. 인덱스 반환 건수 ≠ 최종 건수.
4. 선두 칼럼을 가공하면 INDEX RANGE SCAN이 깨집니다. 인덱스 정렬 순서를 그대로 타야 범위 스캔이 성립합니다.
 - 한 줄 정리 선두 칼럼 등치와 다음 칼럼 범위는 인덱스 액세스 조건이 되고, 인덱스에 없는 칼럼은 테이블 액세스 단계 필터가 되어 Card가 7,400에서 1,850으로 줄어듭니다.

문제 6. 정답 ③

결합 인덱스에서 스캔 범위를 정하는 원리는 하나입니다. 인덱스는 컬럼 순서대로 정렬되어 있으므로, 앞쪽 컬럼이 한 점으로 고정될수록 그 뒤 컬럼들이 정렬된 좁은 구간을 따라 이어 읽을 수 있습니다.

그래서 순서 규칙은 이렇게 갑니다.

등치(=) 조건 컬럼을 앞에 두면 그 컬럼 값이 하나로 고정되어, 뒤 컬럼의 정렬이 그대로 살아 스캔 구간이 좁아집니다.

범위 조건 컬럼이 앞에 오면, 그 컬럼이 여러 값에 걸쳐 열려 버리므로 뒤 컬럼은 구간 안에서 정렬이 흩어져 스캔 시작·끝점을 좁히는 데 기여하지 못하고 필터로만 작동합니다.

③은 이 순서를 정반대로 뒤집었습니다. 범위 조건 컬럼을 앞에, 등치 조건 컬럼을 뒤에 두면 스캔 범위는 최소화되는 게 아니라 오히려 넓어집니다. 올바른 배치는 "등치 조건 컬럼을 앞, 범위 조건 컬럼을 뒤"입니다.

①·②·④는 모두 이 원리의 다른 단면을 옳게 진술합니다. 선두 컬럼의 등치 조건이 범위를 좁히고(①), 선두 컬럼이 빠지면 정상 Range Scan이 어렵고(②), 순서 결정 기준은 저장 크기가 아니라 조건 사용 패턴입니다(④).

선택지	판정	이유
①	○	선두 컬럼이 등치로 고정되면 뒤 컬럼의 정렬이 살아 스캔 시작·끝점이 좁게 확정됩니다. 옳은 서술입니다
②	○	선두 컬럼이 조건에서 빠지면 Skip Scan 등 예외를 빼고는 정상 Range Scan을 쓰기 어렵습니다. 옳은 서술입니다
③	×	방향이 뒤집혔습니다. 범위 컬럼을 앞에 두면 그 뒤 컬럼이 필터로만 작동해 스캔 범위가 넓어집니다. 등치 컬럼을 앞, 범위 컬럼을 뒤에 두어야 좁아집니다
④	○	컬럼 순서는 저장 크기가 아니라 어떤 컬럼이 어떤 연산자로 쓰이는지(조건 사용 패턴)를 기준으로 정합니다

■ 이 문제의 핵심

1. 인덱스는 컬럼 순서대로 정렬됨 → 앞 컬럼이 한 점으로 고정될수록 뒤 컬럼의 정렬이 살아 스캔 구간이 좁아집니다.
2. 그래서 등치(=) 조건 컬럼을 앞, 범위 조건 컬럼을 뒤에. 범위 컬럼이 선두면 그 뒤는 필터로만 작동합니다.
3. 선두 컬럼이 조건에서 누락되면(예외: Skip Scan 등) 정상 Range Scan 활용이 어렵습니다.
4. 컬럼 순서의 판단 기준은 저장 크기가 아니라 조건 사용 패턴(어떤 컬럼이 어떤 연산자로 자주 쓰이는가)입니다.

■ 한 줄 정리 결합 인덱스는 등치 조건 컬럼을 앞, 범위 조건 컬럼을 뒤에 둘 때 스캔 범위가 좁아지며, 이 순서를 뒤집으면 오히려 넓어집니다.

문제 7. 정답 ③

트레이스의 세 숫자를 먼저 나란히 놓습니다.

479,812 ← 인덱스에서 얻은 ROWID 수 (INDEX RANGE SCAN 의 Rows)
 479,500 ← 테이블 액세스 단계의 누적 블록 I/O (TABLE ACCESS 의 cr)
 642 ← 인덱스 스캔에 든 블록 (INDEX RANGE SCAN 의 cr)

클러스터링 팩터는 인덱스 순서로 테이블을 읽을 때 발생하는 블록 I/O 수입니다. 이론상 최솟값은 테이블 블록 수, 최댓값은 테이블 행 수입니다.

CF ≈ 테이블 블록 I/O ÷ ROWID 수 = 479,500 / 479,812 ≈ 1.0
 → ROWID 하나마다 블록을 거의 한 번씩 새로 읽었다는 뜻
 → CF가 행 수에 육박 = 최악에 가까움

즉 ①은 정반대입니다. CF는 매우 나쁩니다.

병목이 어디인지는 **cr** 비교로 끝납니다. 인덱스 스캔은 642 블록, 테이블 액세스는 479,500 블록 — 전체 블록 I/O의 99.9%가 테이블 액세스에서 발생합니다. 인덱스를 손대는 ②는 0.1%를 겨냥하는 것이라 효과가 없습니다(원인 오귀속).

이제 시간을 봅시다.

Elapsed 41.7초 - CPU 8.2초 ≈ 33.5초 ← CPU가 아닌 '기다린' 시간
 Disk = 38,200 블록 ← 물리적으로 디스크에서 읽어야 했던 블록

CPU는 8.2초만 썼는데 실제로는 41.7초가 걸렸습니다. 이 33초의 간극은 일한 시간이 아니라 대기한 시간이고, 그 원인으로 가장 유력한 것은 38,200개 디스크 블록을 읽기 위한 **db file** I/O Call 대기입니다. ④가 지목한 Fetch 횟수(33회)는 Array Size 이슈일 뿐 33초 대기의 원인이 아닙니다.

선택지	판정	이유
①	×	정반대입니다. 테이블 블록 I/O(479,500)가 ROWID 수(479,812)와 거의 같아 CF가 최댓값(행 수)에 육박 — 최악입니다
②	×	인덱스 스캔은 cr=642로 전체(479,500)의 0.1%뿐입니다. 병목은 테이블 액세스이므로 인덱스 재생성은 겨냥점이 틀렸습니다
③	○	CPU 8.2초 대비 Elapsed 41.7초의 33초 간극은 대기 시간이고, 38,200개 디스크 블록 물리 I/O 대기가 가장 유력한 원인입니다
④	×	Fetch 33회는 Array Size에 따른 분할일 뿐입니다. 33초 대기는 Fetch 횟수가 아니라 디스크 I/O에서 비롯됩니다

■ 이 문제의 핵심

1. CF ≈ 테이블 블록 I/O ÷ ROWID 수. 이 값이 1에 가까우면(=행 수에 육박) 최악, 테이블 블록 수에 가까우면 최선입니다.
2. 병목은 **cr** 비율로 지목합니다. 인덱스(642) vs 테이블(479,500) — 99.9%가 테이블 액세스입니다.
3. Elapsed - CPU = 대기 시간. 이 간극은 대개 물리 I/O(**disk**) 대기이지 Fetch 횟수 탓이 아닙니다.
4. 나쁜 CF의 처방은 인덱스 컬럼 순서 조정이 아니라, 인덱스 커버링(테이블 액세스 제거)이나 테이블 재구성입니다.

■ 한 줄 정리 테이블 블록 I/O가 ROWID 수와 맞먹으면 CF는 최악이고, 병목은 **cr** 99.9%를 차지하는 테이블 액세스이며, Elapsed와 CPU의 간극은 디스크 I/O 대기입니다.

문제 8. 정답 ②

②는 "인덱스가 많을수록 조회가 빨라진다"는 반사적 연상을 그대로 결론으로 내세운 함정입니다. 얼핏 그럴듯하지만 두 지점에서 무너집니다.

DML 부하는 인덱스 수에 비례해 늘어납니다. 인덱스는 조회를 돕는 대신, 테이블에 행이 바뀔 때마다 관련 인덱스 엔트리로 모두 갱신되어야 합니다. 인덱스를 "넉넉히" 깔아 두면 INSERT/UPDATE/DELETE 비용이 그만큼 커집니다. 조회가 잦은 테이블이라고 DML이 없는 것은 아닙니다.

쓰이지 않는 인덱스는 조회도 빠르게 하지 못합니다. 한 쿼리는 대개 한두 개의 인덱스만 타며, 나머지 인덱스는 그 쿼리에 아무 이득 없이 저장 공간과 유지 비용, 옵티마이저의 계획 탐색 부담만 더합니다. "개수 $\propto$ 조회 성능"은 성립하지 않습니다.

인덱스 설계는 개수를 늘리는 문제가 아니라, 어느 컬럼을 어떤 순서로 최소한으로 묶을지 고르는 문제입니다. ①(선택도), ④(DML 부하와의 균형), ③(반복 사용 컬럼의 결합)은 모두 그 기준을 옳게 짚었습니다.

선택지	판정	이유
①	○	선택도가 높은(조건을 만족하는 행이 소수인) 컬럼일수록 인덱스로 걸러 내는 효과가 커 액세스 효율이 좋습니다
②	×	"많을수록 조회가 빨라진다"는 반사적 연상입니다. 여분의 인덱스는 DML 부하와 유지 비용만 늘릴 뿐, 인덱스 개수가 조회 성능에 비례하지 않습니다
③	○	반복해 함께 쓰이는 조건 컬럼은 개별 인덱스보다 하나의 결합 인덱스로 묶는 편이 유리한 경우가 많습니다
④	○	인덱스는 조회 성능과 DML 부하의 맞교환입니다. DML이 잦은 테이블일수록 인덱스 개수를 신중히 관리해야 합니다

■ 이 문제의 핵심

1. 인덱스는 조회 성능 $\leftrightarrow$ DML 부하의 맞교환입니다. 개수를 늘리면 조회가 무조건 좋아진다는 발상이 함정입니다.
2. 한 쿼리는 소수의 인덱스만 탑니다. 쓰이지 않는 인덱스는 이득 없이 저장·유지·탐색 비용만 더합니다.
3. 좋은 설계 기준: 선택도가 높은 컬럼, 반복해 함께 쓰이는 컬럼의 결합 인덱스화, 그리고 최소한의 개수.
4. 설계의 본질은 "몇 개 더 만들까"가 아니라 "어떤 컬럼을 어떤 순서로 꼭 필요한 만큼 묶을까"입니다.

■ 한 줄 정리 인덱스는 많이 만들수록 조회가 빨라지는 것이 아니라 DML 부하만 늘 뿐이며, 설계의 핵심은 선택도·결합·최소 개수입니다.

문제 9. 정답 ②

NL 조인의 동작을 한 줄로 요약하면 이렇습니다. 드라이빙(outer) 테이블에서 한 건을 읽을 때마다, 그 값으로 inner 테이블을 한 번씩 반복 탐색합니다. 즉 inner 테이블을 찾아가는 횟수는 드라이빙 테이블에서 읽히는 행 수가 그대로 결정합니다.

드라이빙에서 읽히는 행 수 = inner 반복 탐색 횟수

그러므로 드라이빙 테이블로는 조인 결과에 참여하는 행이 적은(선택도가 좋아 먼저 걸러지는) 작은 집합을 앞세워 반복 횟수를 줄이는 것이 유리합니다.

②은 이 기준을 정반대로 진술했습니다. "참여 행이 많은 큰 집합을 드라이빙으로 고르면 반복 횟수가 줄어 유리하다"는 것은 방향이 뒤집힌 서술입니다. 드라이빙이 클수록 inner를 찾아가는 횟수가 늘어나 오히려 불리해집니다.

①·③·④는 옳습니다. inner는 조인 컬럼 인덱스를 통한 Random 액세스 처리되고(①), 그 인덱스가 없으면 반복마다 테이블을 훑어 반복이 많을수록 급격히 나빠지며(③), NL은 드라이빙을 읽는 순서대로 결과가 나와 부분범위처리에 유리합니다(④).

선택지	판정	이유
①	○	inner는 드라이빙 값으로 반복 탐색되며, 조인 컬럼 인덱스가 있으면 ROWID를 통한 Random 액세스 효율적으로 처리됩니다
②	×	방향이 뒤집혔습니다. inner 반복 횟수는 드라이빙에서 읽히는 행 수가 결정하므로, 큰 집합을 드라이빙으로 삼으면 반복이 늘어 불리합니다. 작은(선택도 좋은) 집합을 앞세워야 합니다
③	○	inner 조인 컬럼에 인덱스가 없으면 드라이빙 건마다 테이블을 훑어야 하므로, 반복 횟수가 많을수록 비용이 급격히 커집니다
④	○	NL 조인은 드라이빙을 읽는 순서대로 결과를 흘려보내므로, 앞쪽 일부만 필요한 부분범위처리에 유리합니다

■ 이 문제의 핵심

1. NL 조인의 반복 횟수 = 드라이빙 테이블에서 읽히는 행 수. 그래서 드라이빙은 작고 선택도 좋은 집합을 앞세웁니다.

2. inner 테이블은 조인 컬럼 인덱스를 통한 Random 액세스로 반복 탐색됩니다. 이 인덱스가 성능의 관건입니다.
3. inner 조인 컬럼에 인덱스가 없으면 반복마다 테이블 스캔 → 반복이 많을수록 급격히 악화됩니다.
4. NL은 드라이빙 순서대로 결과를 내보내므로 부분범위처리·OLTP 소량 조회에 강합니다.
 - 한 줄 정리 NL 조인의 드라이빙 테이블은 반복 횟수를 줄이도록 작고 선택도 좋은 집합을 골라야 하며, 큰 집합을 드라이빙으로 삼는다는 서술은 방향을 뒤집은 것입니다.

문제 10. 정답 ④

해시 조인은 두 단계로 움직입니다.

Build 단계: 한쪽 입력(Build Input)을 읽어 조인 키로 해시 테이블을 메모리(Hash Area)에 구축합니다.

Probe 단계: 다른 쪽 입력(Probe Input)을 한 행씩 읽어 같은 해시 함수로 버킷을 찾아 매칭합니다.

이 구조에서 비용을 좌우하는 것은 해시 테이블이 메모리에 통째로 들어가느냐입니다. 그래서 옵티마이저는 두 입력 중 더 작을 것으로 추정되는 쪽을 Build Input으로 잡습니다. 작은 쪽을 메모리에 올릴수록 해시 테이블이 Hash Area를 넘지 않을 가능성이 커지고, 디스크(Temp)로 넘치는 일(one-pass·multi-pass)을 피할 수 있기 때문입니다. ④가 이 원리를 정확히 진술합니다.

나머지는 해시 조인의 경계를 뭉개진술입니다.

해시 조인은 해시 함수로 같은 값을 같은 버킷에 모으는 방식이라, 등치 조건에서만 성립합니다(①). 범위·부등호 조인은 해시로 버킷을 특정할 수 없습니다.

Probe를 시작하려면 Build Input을 끝까지 다 읽어 해시 테이블을 완성해야 합니다. 즉 첫 결과가 나오기 전에 한쪽 전체를 소진하므로, 부분범위 처리(빠른 첫 행)와는 오히려 상극입니다(③).

Build Input이 메모리를 넘으면 실패가 아니라 Temp 영역으로 분할(one-pass/multi-pass)해 계속 수행합니다(②). 느려질 뿐 오류가 아닙니다.

선택지	판정	이유
①	×	해시 조인은 조인 키를 해시 버킷에 모으는 방식이라 등치(=) 조건에서만 성립합니다. 부등호·범위 조인에는 사용할 수 없습니다
②	×	메모리를 넘치면 오류로 끝나지 않고 Temp 영역으로 분할해 수행합니다. 성능이 저하될 뿐 조인은 완료됩니다
③	×	Probe 이전에 Build Input을 끝까지 읽어 해시 테이블을 완성해야 첫 행이 나옵니다. 부분범위 처리에 적합하다는 것은 경계를 뭉개진술 오해입니다
④	○	작은 쪽을 Build Input으로 삼으면 해시 테이블이 Hash Area에 들어갈 가능성이 커져 Temp 분할(one-pass/multi-pass)을 피할 수 있습니다

▪ 이 문제의 핵심

1. Build → Probe 2단계. Build Input으로 해시 테이블을 만들고, Probe Input을 훑어 매칭합니다.
2. 작은 쪽이 Build Input. 메모리(Hash Area)에 올릴 해시 테이블을 작게 유지해 Temp 분할을 피하려는 선택입니다.
3. 등치 조건 전용. 해시는 같은 값을 같은 버킷에 모으므로 부등호·범위 조인에는 못 씁니다.
4. 부분범위 처리와 상극. Build Input을 다 읽어야 첫 행이 나오므로, 빠른 첫 행 응답이 필요한 상황에는 부적합합니다.
 - 한 줄 정리 해시 조인은 작은 쪽을 Build Input으로 해시 테이블을 메모리에 짓고 큰 쪽으로 Probe하는 등치 조인 기법이며, 한쪽을 다 읽어야 첫 행이 나오므로 부분범위 처리에는 맞지 않습니다.

문제 11. 정답 ②

NESTED LOOPS(Id 1) 아래 두 자식의 위치를 읽습니다.

```

Id 1  NESTED LOOPS
Id 2  TABLE ACCESS FULL          주문          ← 먼저 오는 자식 = 드라이빙(선행)
Id 3  TABLE ACCESS BY INDEX ROWID 주문상세      ← 나중 자식 = 인너(후행)
Id 4  INDEX RANGE SCAN            주문상세_IX     ← 인너 접근 경로
  
```

NESTED LOOPS에서 먼저 나열된 자식이 드라이빙(선행) 집합입니다. 여기서는 주문이 TABLE ACCESS FULL로 한 번 훑어지고, 그 각 행마다 인너인 주문상세를 주문상세_IX INDEX RANGE SCAN으로 반복 탐색합니다. 인덱스가 주문번호 단일 칼럼이고 주문:주문상세가 1:N이므로, 한 주문번호에 여러 상세가 걸려 UNIQUE가 아닌 RANGE SCAN으로 나타납니다.

그래서 방향은 주문 → 주문상세이고, 인너 스캔은 드라이빙 건수만큼 반복됩니다. ②이 이 구조 그대로입니다.

- ①은 드라이빙/인너를 거꾸로 지목합니다. 먼저 나온 자식이 주문이므로 드라이빙은 주문입니다.
- ③는 이 플랜에 없는 SORT MERGE 조인을 끌어옵니다. 플랜에는 정렬 노드가 없고 TABLE ACCESS BY INDEX ROWID(랜덤 액세스)가 분명히 있습니다.
- ④는 이 플랜에 없는 HASH JOIN을 끌어옵니다. 주문상세는 TABLE ACCESS FULL이 아니라 인덱스로 접근하며, HASH 노드도 없습니다.

선택지	판정	이유
①	×	드라이빙/인너를 뒤집었습니다. NESTED LOOPS의 먼저 나온 자식(Id 2, 주문)이 드라이빙이며, 주문상세가 인너입니다
②	○	주문(Id 2)이 드라이빙, 주문상세가 인너이며 주문 건마다 주문상세_IX RANGE SCAN(Id 4)이 반복됩니다
③	×	정렬-병합 노드가 플랜에 없고, Id 3의 TABLE ACCESS BY INDEX ROWID가 랜덤 액세스를 나타냅니다. SORT MERGE는 이 플랜 밖의 개념입니다
④	×	HASH JOIN·해시 영역 적재 노드가 없고, 주문상세는 FULL이 아니라 INDEX RANGE SCAN으로 접근합니다. 이 플랜 밖의 조인 방식입니다

▪ 이 문제의 핵심

- NESTED LOOPS의 먼저 나열된 자식이 드라이빙(선행) 집합입니다. 방향은 위→아래로 읽습니다.
 - 인너 테이블 접근은 드라이빙 건수만큼 반복됩니다. 여기서는 **주문상세_IX** INDEX RANGE SCAN이 주문 행마다 재실행됩니다.
 - 1:N 조인 키에는 UNIQUE가 아닌 RANGE SCAN이 나타납니다. **주문번호** 하나에 상세 여러 건이 대응하기 때문입니다.
 - 정렬 노드·해시 노드의 유무로 조인 방식을 판별합니다. 둘 다 없으면 NL, 있으면 각각 SORT MERGE·HASH입니다.
- 한 줄 정리 NESTED LOOPS에서 위쪽 자식 주문이 드라이빙이고, 주문 건마다 **주문상세_IX** INDEX RANGE SCAN이 반복되어 인너인 주문상세가 조인됩니다.

문제 12. 정답 ②

CBO에는 서로 독립인 두 축이 있습니다.

- 축 A) 계획의 '품질' ← 통계가 정확할수록 카디널리티 추정이 맞아 좋은 계획을 고름
 축 B) 파싱의 '비용' ← 실행계획을 탐색·평가하는 데 드는 시간(하드파싱 부하)

- ①④③은 모두 축 A를 정확히 설명합니다. 통계는 CBO가 각 계획의 비용을 계산하는 재료이고(①), 통계가 실제 분포와 어긋나면 카디널리티 추정이 틀어져 나쁜 계획으로 이어지며(④), 히스토그램은 편중된 분포에서 선택도 추정을 정밀하게 해 줍니다(③).
- ②는 이 축 A(품질)와 축 B(파싱 비용)를 인과로 엮은 진술입니다. 통계가 정확하다고 해서 옵티마이저의 계획 탐색량이 줄어들지는 않습니다. 오히려 히스토그램·확장 통계가 많아지면 평가할 정보가 늘어 하드파싱 부하는 커지는 경향도 있습니다. 통계 정확도(계획이 좋아짐)와 파싱 시간(계획을 찾는 비용)은 별개의 축인데, ②는 "정확 → 파싱이 빨라짐"이라는 없는 인과를 만들어 냈습니다. 그래서 ②가 가장 부적절합니다.

선택지	판정	이유
①	○	통계정보(행 수·블록 수·NDV·히스토그램)로 각 후보의 예상 비용을 산정하고 최소 비용 계획을 고르는 것이 CBO의 정의입니다
②	×	통계 정확도(계획 품질)와 파싱 시간(계획 탐색 비용)은 독립 축입니다. 통계가 정확해진다고 하드파싱이 빨라지지 않으며, 히스토그램이 늘면 오히려 파싱 부하가 커질 수 있습니다
③	○	값 분포가 편중된 컬럼에서 히스토그램은 선택도 추정 정밀도를 높입니다
④	○	통계가 실제 분포와 어긋나면 카디널리티 추정이 틀어져 비효율 계획으로 이어질 수 있습니다 — 통계 정확도가 계획 품질을 좌우하는 축입니다

▪ 이 문제의 핵심

- CBO = 통계로 비용을 매겨 최소 비용 계획을 선택하는 옵티마이저입니다.
- 통계 정확도는 '계획 품질' 축입니다. 틀린 통계 → 틀린 카디널리티 → 나쁜 계획.
- 파싱 시간은 별개 축입니다. 계획을 찾고 평가하는 데 드는 비용이지, 통계 정확도의 함수가 아닙니다.
- 두 축을 "정확하면 파싱이 빨라진다"로 잇는 것은 전형적인 별개 축 혼동입니다 — 오히려 히스토그램이 많아지면 파싱 부하가 늘 수 있습니다.

- 한 줄 정리 통계 정확도는 계획의 품질을 높이는 축이고 파싱 시간은 그와 독립인 비용 축이므로, "통계가 정확할수록 하드파싱이 빨라진다"는 없는 인과를 만든 부적절한 진술입니다.

문제 13. 정답 ④

라이브러리 캐시 요약이 답을 그대로 보여 줍니다. 논리적으로 같은 세 SQL인데 SQL_ID가 세 개(9k2m..., 7f3p..., 2d5x...)로 갈렸고, 각각 **PARSE_CALLS 1 / EXECUTIONS 1**입니다.

9k2m... ← 세션 A : 원본
7f3p... ← 세션 B : SELECT 만 대문자
2d5x... ← 세션 C : from 앞 공백 2칸 + 끝 주석
→ 텍스트가 다르면 SQL_ID가 다르고, 커서를 공유하지 못해 각각 하드파싱

오라클의 기본(EXACT) 커서 공유는 SQL 텍스트를 문자 단위로 비교합니다. 대소문자·공백·주석 하나만 달라도 다른 텍스트로 간주되어 라이브러리 캐시에서 기존 커서를 찾지 못하고, 새 커서를 만들며 하드파싱합니다. 요약표의 세 SQL_ID가 바로 그 결과입니다. ④이 정확합니다.

②는 "옵티마이저가 텍스트를 정규화(대문자화·여백/주석 제거)해 공유한다"는 정규화 가정의 오류입니다. 기본 모드에서 오라클은 그런 정규화를 하지 않으므로, 세 SQL은 공유되지 않습니다(요약표가 SQL_ID 3개로 반증).

③은 오히려 반대입니다. 리터럴(상수 100)을 그대로 쓰면 값이 바뀔 때마다 텍스트가 달라져 커서 공유가 깨집니다. 커서를 공유하려면 상수를 바인드 변수(:b1)로 바꿔 텍스트를 동일하게 만들어야 합니다. "같은 상수라 자동 공유"는 성립하지 않습니다.

①의 주석은 실행계획 결과에는 영향이 없더라도 SQL 텍스트의 일부입니다. 세션 C처럼 주석이 붙으면 텍스트가 달라져 SQL_ID가 갈립니다(2d5x...). 따라서 "주석만 달라도 같은 커서 공유"는 틀립니다.

선택지	판정	이유
①	×	주석은 SQL 텍스트의 일부라 SQL_ID를 바꿉니다. 세션 C의 별도 SQL_ID(2d5x...)가 그 증거입니다
②	×	기본 커서 공유는 텍스트를 대문자화·여백/주석 제거로 정규화하지 않습니다. 정규화된다면 SQL_ID가 하나여야 하는데 표에는 셋입니다
③	×	같은 상수를 써도 공유가 자동으로 되는 것이 아닙니다. 오히려 리터럴을 바인드 변수로 바꿔야 텍스트가 같아져 공유됩니다
④	○	세 SQL은 의미가 같아도 텍스트가 문자 단위로 다릅니다. 요약표의 SQL_ID가 셋(9k2m·7f3p·2d5x)으로 갈려 각각 하드파싱되었음을 보여 줍니다

▪ 이 문제의 핵심

1. 기본 커서 공유는 텍스트를 문자 단위로 비교합니다. 대소문자·공백·주석이 하나만 달라도 다른 SQL_ID입니다.
2. 텍스트가 다르면 = 하드파싱. 요약표의 SQL_ID 개수가 곧 하드파싱한 커서 개수입니다.
3. 리터럴 대신 바인드 변수. 값이 바뀌어도 텍스트가 같아져 커서를 공유(소프트파싱)할 수 있습니다.
4. 옵티마이저가 텍스트를 정규화해 공유해 준다는 것은 흔한 착각입니다 — 기본 모드에서는 그런 정규화가 없습니다.

- 한 줄 정리 오라클 기본 커서 공유는 SQL 텍스트를 문자 단위로 비교하므로 대소문자·공백·주석만 달라도 각각 하드파싱되며, 공유하려면 리터럴을 바인드 변수로 바꿔 텍스트를 동일하게 만들어야 합니다.

문제 14. 정답 ①

쿼리 변환의 공통 목적은 옵티마이저가 더 넓은 계획 공간을 탐색하도록 문장 구조를 바꾸는 것입니다.

View Merging은 뷰의 정의를 바깥 쿼리에 풀어 넣어 하나의 쿼리 블록으로 합치는 변환입니다. 합쳐지면 뷰 경계가 사라져, 옵티마이저가 뷰 안팎의 테이블을 자유롭게 조인 순서에 배치하고 각 테이블의 액세스 경로도 폭넓게 고를 수 있습니다. ①가 이 원리를 정확히 진술합니다.

나머지는 키워드의 반사적 연상을 노린 함정입니다.

②: "Unnesting"이라는 단어에서 "중첩(nested)"을 반사적으로 떠올리게 하지만, 방향이 반대입니다. Unnesting은 서브쿼리를 조인으로 풀어 옵티마이저의 선택지를 늘리는 변환이지, 조인을 서브쿼리로 되돌리거나 NL을 강제하는 것이 아닙니다.

③: 조건절 Pushing은 이름 그대로 조건을 안쪽(뷰)으로 밀어 넣어 일찍 걸러 내는 기법입니다. 머지되지 못하는 뷰에 바깥 조건을 밀어 넣어 뷰가 처리할 데이터를 줄입니다. "바깥으로 끌어올려 마지막에 필터링"은 방향이 정반대라 이득이 사라집니다.

④: 복합 View Merging은 GROUP BY·DISTINCT가 있는 뷰도 조건이 맞으면 머지할 수 있습니다. "GROUP BY 뷰는 머지 불가"라는 단정은 반쪽 사실을 전체로 굳힌 오류입니다.

선택지	판정	이유
①	○	View Merging은 뷰와 바깥 쿼리를 한 블록으로 합쳐 조인 순서·액세스 경로 선택의 폭을 넓혀 줍니다
②	×	Unnesting은 서브쿼리를 조인으로 푸는 변환입니다. "조인을 서브쿼리로 되돌려 NL을 강제"는 방향이 반대이며, 'nested'라는 단어의 반사적 연상을 노린 함정입니다
③	×	Pushing은 조건을 뷰 안으로 밀어 넣어 일찍 거르는 기법입니다. 바깥으로 끌어올려 마지막에 필터링한다는 것은 방향이 반대라 조기 필터링 이득이 사라집니다
④	×	복합 View Merging은 GROUP BY·DISTINCT 뷰도 조건이 맞으면 머지합니다. "머지 불가"라는 단정은 성립하지 않습니다

▪ 이 문제의 핵심

1. 쿼리 변환의 목적은 계획 공간 확장입니다 — 옵티마이저가 더 많은 조인 순서·경로를 보게 만듭니다.
2. View Merging = 뷰를 바깥에 합치기. 뷰 경계를 없애 조인 순서·액세스 경로 선택을 자유롭게 합니다.
3. Unnesting = 서브쿼리를 조인으로 풀기. 'nested'라는 단어에 이끌려 방향을 반대로 읽지 않도록 주의합니다.
4. Pushing = 조건을 안으로 밀어 조기 필터링. 밖으로 끌어올리는 것이 아닙니다. GROUP BY 복합 뷰도 머지될 수 있습니다.

▪ 한 줄 정리 View Merging은 뷰와 바깥 쿼리를 한 블록으로 합쳐 조인 순서·경로 선택의 폭을 넓히는 변환이며, Unnesting은 서브쿼리를 조인으로 풀고 Pushing은 조건을 안으로 밀어 일찍 거르는 기법입니다.

문제 15. 정답 ③

Direct Path Insert(/** append */)의 특징을 하나씩 대응시키면 ③만 방향이 뒤집혀 있습니다.

①(참): append는 데이터를 버퍼 캐시에 담지 않고 세그먼트 HWM 위쪽 새 블록에 직접(Direct Path) 기록합니다. 버퍼 캐시를 거치지 않으니 캐시 경합·오버헤드가 줄어듭니다.

②(참): 대상 테이블이 **nologging**이고 direct path(append)가 적용되면, 삽입 데이터에 대한 Redo가 최소화됩니다(공간·딕셔너리 변경 정도만 로깅). 대량 적재 속도를 높이는 핵심 효과입니다.

④(참): direct path는 HWM 아래의 여유 공간(삭제로 빈 블록)을 재사용하지 않고 HWM 위에만 붙여 씁니다. 그래서 반복하면 세그먼트가 실제 데이터량보다 커질 수 있습니다.

③(거짓 — 정반대 진술): append(Direct Path Insert)는 오히려 대상 테이블에 배타적 TM Lock(Exclusive)을 획득합니다. 그 결과 커밋 전까지 다른 세션은 같은 테이블에 INSERT·UPDATE·DELETE를 수행할 수 없습니다. ③은 "Lock을 걸지 않아 동시 INSERT가 가능하다"라며 사실을 정확히 뒤집었습니다. Direct Path의 대표적 제약(동시 DML 차단)을 반대로 진술했으므로 가장 부적절합니다.

선택지	판정	이유
①	○	append는 버퍼 캐시를 우회해 HWM 위 새 블록에 직접 기록하므로 버퍼 캐시 경합이 줄어듭니다
②	○	nologging 테이블에 direct path가 적용되면 삽입 데이터의 Redo가 최소화됩니다(공간·딕셔너리 변경만 로깅)
③	×	방향이 반대입니다. append는 대상 테이블에 배타적 TM Lock을 걸어, 커밋 전까지 다른 세션의 동시 DML(INSERT 포함)을 차단합니다
④	○	HWM 아래 여유 공간을 재사용하지 않고 HWM 위에 붙여 쓰므로 세그먼트가 커질 수 있습니다

▪ 이 문제의 핵심

1. Direct Path(append) = 버퍼 캐시 우회 + HWM 위 직접 기록. 캐시 경합이 줄고 대량 적재가 빠릅니다.
2. nologging + append → Redo 최소화. 단, 복구를 위해서는 이후 백업 전략을 고려해야 합니다.
3. 여유 공간 미재사용. HWM 아래 빈 블록을 건너뛰고 위에만 쓰므로 세그먼트가 부풀 수 있습니다.
4. 배타적 TM Lock으로 동시 DML 차단. 커밋 전까지 같은 테이블에 다른 세션의 INSERT/UPDATE/DELETE가 막힙니다 — 이 제약을 반대로 읽지 않도록 주의합니다.

▪ 한 줄 정리 Direct Path Insert는 버퍼 캐시를 우회해 HWM 위에 직접 기록하고 nologging과 함께 Redo를 최소화하지만, 대상 테이블에 배타적 TM Lock을 걸어 커밋 전까지 동시 DML을 차단합니다.

문제 16. 정답 ①

Partition Pruning은 옵티마이저가 WHERE 조건의 값을 파티션 경계(VALUES LESS THAN)와 대조해 스캔 대상 파티션을 미리 좁히는 최적화입니다. 이 대조가 가능하려면 조건이 파티션 키 칼럼(매출일자) 자체에 직접 걸려 있어야 합니다.

- ④ 매출일자 ∈ [2026-03-01, 2026-04-01] → p_202603 한 개
- ② 매출일자 ∈ [2026-02-01, 2026-02-28] → p_202602 한 개
- ③ 매출일자 ∈ [2026-05-01, 2026-07-01] → p_202605, p_202606 두 개
- ① TO_CHAR(매출일자, 'YYYYMM') = '202603' → 키에 함수, 경계 대조 불가

④②③은 모두 매출일자를 가공하지 않은 날짜 범위로 걸었으므로, 옵티마이저가 경계와 대조해 해당 파티션만 접근합니다(③은 경계상 두 파티션이지만 여전히 나머지는 건너뛴다 — Pruning 작동).

①는 **TO_CHAR(매출일자, ...)**로 파티션 키에 형변환 함수를 씌웠습니다. 그러면 옵티마이저는 함수 결과값을 파티션 경계와 대조할 수 없어 Pruning을 포기하고, **p_202601 ... p_max** 모든 파티션을 스캔한 뒤 각 행에서 TO_CHAR를 평가해 필터합니다. 조건이 의미상 "2026년 3월"을 겨냥하는 부분적 진실이라도, 형변환 한 줄이 최선(1개 파티션)을 최악(전 파티션 스캔)으로 뒤집습니다. 그래서 Pruning이 효과적으로 작동한다고 볼 수 없는 것은 ①입니다.

선택지	판정	이유
①	×	파티션 키에 TO_CHAR 를 씌워 경계 대조가 불가능해집니다. 전 파티션을 스캔한 뒤 필터하므로 Pruning이 깨집니다
②	○	BETWEEN 구간이 2026-02 경계 안에 들어가 p_202602 한 개만 접근합니다. Pruning 작동합니다
③	○	두 달 범위라 p_202605·p_202606 두 파티션을 읽지만, 나머지 파티션은 건너뛰므로 Pruning은 여전히 작동합니다
④	○	매출일자를 가공 없이 [2026-03-01, 2026-04-01] 범위로 걸어 p_202603 한 개로 정확히 Pruning됩니다

■ 이 문제의 핵심

1. Partition Pruning은 파티션 키 칼럼에 직접 걸린 조건에서만 작동합니다. 키를 가공하면 경계와 대조할 수 없습니다.
2. 날짜 범위 조건(>=, <, BETWEEN)은 경계와 그대로 대조되어 필요한 파티션만 남깁니다.
3. 여러 파티션에 걸치더라도(③) Pruning은 유효합니다 — 나머지 파티션을 건너뛰기만 하면 됩니다.
4. **TO_CHAR·TRUNC** 등 키 가공은 Pruning을 무력화해 전 파티션 스캔을 유발합니다. "월만 뽑았다"는 부분적 타당함이 함정입니다.

■ 한 줄 정리 파티션 키를 가공하지 않은 날짜 범위는 Pruning이 작동하지만, **TO_CHAR(매출일자, ...)**처럼 키에 형변환을 씌우면 경계 대조가 불가능해져 전 파티션을 스캔합니다.

문제 17. 정답 ①

분배 방식은 각 생산자 TQ의 PX SEND 종류로 읽습니다.

Id 5 PX SEND HASH :TQ10000 거래이력 쪽 생산자 → HASH 재분배
 Id 9 PX SEND HASH :TQ10001 계좌 쪽 생산자 → HASH 재분배
 Id 3 HASH JOIN BUFFERED (:TQ10002) 두 스트림을 받아 조인

Id 5는 거래이력(Id 7)을 읽어 PX SEND HASH로 내보냅니다 → 거래이력을 조인 키 해시로 재분배.

Id 9는 계좌(Id 11)를 읽어 PX SEND HASH로 내보냅니다 → 계좌도 조인 키 해시로 재분배.

양쪽 모두 IN-OUT이 **P→P**(병렬→병렬)이고, **PX SEND BROADCAST**는 어디에도 없습니다.

두 입력이 같은 해시 함수로 재분배되므로, 같은 조인 키는 같은 병렬 서버로 모입니다. 각 서버는 자신이 맡은 해시 버킷 범위 안에서만 양쪽 행을 만나 조인합니다. 이것이 hash-hash 분배이고 ①의 서술 그대로입니다.

나머지 셋은 실제 노드와 값을 바꿔치기한 것입니다.

실제: 거래이력 = HASH, 계좌 = HASH (① hash-hash)
 ② broadcast-none : 거래이력 = BROADCAST, 계좌 = 재분배없음 → SEND HASH 두 개와 불일치
 ④ hash-broadcast : 거래이력 = HASH, 계좌 = BROADCAST → Id 9가 SEND HASH이므로 불일치
 ③ QC 단일서버 : 재분배 없음, 단일 서버 조인 → P→P·PX SEND HASH 두 개와 불일치

선택지	판정	이유
①	○	Id 5·Id 9가 둘 다 PX SEND HASH (P->P)이고 BROADCAST가 없습니다. 양쪽을 조인 키 해시로 재분배하는 hash-hash 분배입니다
②	×	broadcast 노드가 없습니다. 거래이력 쪽 Id 5는 BROADCAST가 아니라 PX SEND HASH 이고, 계좌도 재분배(Id 9)됩니다
③	×	두 생산자 TQ가 P->P 로 병렬 재분배 중이며 조인(Id 3)도 병렬(PCWP)입니다. QC 취합·단일 서버 조인이 아닙니다
④	×	계좌 쪽 Id 9가 PX SEND HASH 이므로 broadcast가 아닙니다. 한쪽만 해시라는 hash-broadcast로 값을 뒤바꾼 진술입니다

▪ 이 문제의 핵심

1. 분배 방식은 PX SEND의 종류로 판별합니다 — **PX SEND HASH**면 해시 재분배, **PX SEND BROADCAST**면 복제입니다.
 2. 양쪽 생산자 TQ가 모두 SEND HASH이고 P->P면 hash-hash 분배입니다. 같은 키가 같은 서버로 모여 버킷 단위로 조인됩니다.
 3. broadcast-none·hash-broadcast는 한쪽에 BROADCAST 노드가 있어야 성립합니다. 이 플랜엔 없습니다.
 4. **P->P**(병렬→병렬) IN-OUT은 재분배가 일어남을 뜻합니다. QC로 모아 단일 서버에서 조인하는 구조가 아닙니다.
- 한 줄 정리 거래이력·계좌 양쪽 생산자 모두 **PX SEND HASH**(P->P)로 내보내고 BROADCAST 노드가 없으므로, 두 테이블을 조인 키 해시로 재분배하는 hash-hash 분배입니다.

문제 18. 정답 ③

UNION과 UNION ALL의 결정적 차이는 중복 제거입니다.

UNION은 두 집합을 합친 뒤 중복을 제거합니다. 이 중복 제거를 위해 내부적으로 정렬(Sort) 또는 해싱이 필요합니다.

UNION ALL은 중복을 제거하지 않고 그대로 이어 붙입니다. 따라서 정렬/해싱 단계가 없습니다.

두 집합이 애초에 중복 행이 없다고 보장되면, **UNION**으로 얻는 결과와 **UNION ALL**로 얻는 결과가 같습니다. 그렇다면 굳이 비용이 드는 중복 제거(정렬·해싱)를 수행하는 **UNION**을 쓸 이유가 없습니다. 이때 **UNION ALL**로 바꾸면 동일한 결과를 더 적은 비용으로 얻습니다. ③가 정확합니다.

나머지는 부분 사실을 전제로 부풀리거나 방향을 뒤집은 진술입니다.

- ①: 분석함수는 GROUP BY와 달리 상세 행을 그대로 유지한 채 각 행 옆에 집계·순위 값을 덧붙입니다. "상세 행을 유지할 수 없다"는 반대입니다.
- ②: 분석함수는 **OVER()** 절로 윈도우를 정의하며 GROUP BY 없이 동작합니다. GROUP BY를 강제한다는 것은 성립하지 않습니다.
- ④: **UNION ALL**의 건수가 "두 집합의 합"인 것은 맞지만(부분 사실), 이를 중복을 제거하는 **UNION**에까지 그대로 확장한 것이 오류입니다. **UNION**은 중복을 지우므로 결과 건수가 두 집합의 합 이하입니다.

선택지	판정	이유
①	×	분석함수는 상세 행을 유지한 채 집계·순위 값을 덧붙입니다. 그룹당 한 행으로 압축하는 것은 GROUP BY의 동작입니다
②	×	분석함수는 OVER() 로 윈도우를 정의하며 GROUP BY 없이 동작합니다. GROUP BY를 반드시 동반한다는 것은 사실이 아닙니다
③	○	두 집합에 중복이 없으면 UNION과 UNION ALL의 결과가 같으므로, 중복 제거 정렬/해싱을 생략하는 UNION ALL로 같은 결과를 더 싸게 얻습니다
④	×	"UNION ALL 건수 = 두 집합 합"이라는 부분 사실을 UNION에 그대로 확장한 오류입니다. UNION은 중복을 제거하므로 건수가 두 집합 합 이하입니다

▪ 이 문제의 핵심

1. UNION은 중복 제거(정렬/해싱) 비용이 든다. UNION ALL은 그냥 이어 붙일 뿐이라 그 단계가 없습니다.
2. 중복이 없으면 UNION → UNION ALL. 결과가 같으면서 정렬/해싱을 생략해 더 저렴합니다.
3. 분석함수는 상세 행을 유지한 채 집계·순위를 덧붙입니다. GROUP BY처럼 행을 압축하지 않고, GROUP BY를 동반하지도 않습니다.

4. 부분 사실을 전체로 부풀리지 말 것. UNION ALL의 건수 등식(=합)은 중복을 지우는 UNION에는 그대로 적용되지 않습니다.

- 한 줄 정리 두 집합에 중복이 없으면 UNION 대신 UNION ALL로 정렬/해싱을 생략해 같은 결과를 더 싸게 얻을 수 있고, 분석함수는 상세 행을 유지한 채 집계·순위를 덧붙이므로 GROUP BY와 다릅니다.

문제 19. 정답 ①

블로킹을 판별하는 자리는 동일한 자원(resource)에 대해 한쪽은 GRANT, 다른 쪽은 WAIT인 행입니다. 표를 자원별로 갈라 보면 답이 한 곳에서만 나옵니다.

자원	53의 상태	57의 상태	호환 여부
OBJECT 계좌 (IX / IX)	GRANT	GRANT	IX↔IX 호환 → 대기 없음
PAGE 1:8842 (IX / IX)	GRANT	GRANT	IX↔IX 호환 → 대기 없음
KEY 100-2277 (X / X)	GRANT	WAIT	X↔X 비호환 → 여기서 대기

OBJECT와 PAGE의 IX(Intent Exclusive)는 서로 호환됩니다. 두 세션이 같은 테이블·페이지에 IX를 나란히 GRANT받은 것은 정상이며 블로킹의 근거가 아닙니다.

실제 충돌은 KEY 자원에서 일어납니다. 세션 53이 계좌번호 100-2277 행에 X 락을 GRANT로 잡았고, 세션 57이 같은 키에 X를 요청했으나 status가 WAIT입니다. X↔X는 비호환이므로 57은 53의 커밋/롤백을 기다립니다.

따라서 블로커는 53, 피해자는 57이고, 그 근거는 IX가 아니라 KEY 행의 WAIT 상태입니다. ①이 방향과 근거를 모두 맞혔습니다.

선택지	판정	이유
①	○	KEY 100-2277에서 53은 X GRANT, 57은 X WAIT입니다. 동일 자원의 GRANT↔WAIT 쌍이 블로킹의 유일한 판별 근거이며, 57이 53에 막혀 있습니다
②	×	방향이 반대입니다. 세션 53의 세 행은 모두 GRANT여서 기다리는 쪽이 아닙니다. 대기(WAIT)는 57에만 있으므로 피해자는 57입니다
③	×	두 세션이 함께 가진 OBJECT의 IX는 서로 호환되는 락입니다. 공통으로 보이는 IX를 블로킹 근거로 삼으면 안 됩니다 — 상호 블로킹이 아닙니다
④	×	PAGE 1:8842는 두 세션 모두 IX GRANT여서 경합이 없습니다. WAIT는 57 한쪽에만 있어 상호 대기(교착)도 아닙니다

▪ 이 문제의 핵심

1. 블로킹 판별은 request_status로 합니다. 같은 자원 행에서 GRANT와 WAIT가 마주칠 때, WAIT 쪽이 GRANT 쪽에 막힌 것입니다.
2. IX(Intent) 락끼리는 호환됩니다. 여러 세션이 같은 OBJECT·PAGE에 IX를 함께 보유하는 것은 정상이며 블로킹 신호가 아닙니다.
3. 모두에게 공통으로 보이는 단서(둘 다 IX 보유)는 판별 근거가 될 수 없습니다 — 차이가 나는 지점(KEY의 X GRANT vs X WAIT)이 근거입니다.
4. 대기가 한쪽에만 있으면 단방향 블로킹이지 교착이 아닙니다. 교착은 양쪽 모두 WAIT일 때입니다.

- 한 줄 정리 공통으로 가진 IX가 아니라 동일 KEY 자원의 X GRANT(53) ↔ X WAIT(57) 쌍이 블로킹의 근거이며, 57이 53에 막혀 있습니다.

문제 20. 정답 ④

두 DBMS의 기본 격리수준은 모두 READ COMMITTED로 같지만, "읽기가 쓰기를 만났을 때"의 처리 방식이 다릅니다. 오라클은 MVCC(다중 버전)로 읽고, 기본 SQL Server는 공유 잠금(locking)으로 읽습니다.

[오라클] - MVCC, 읽기는 쓰기를 기다리지 않음

t1: TX1이 500 → 380 갱신(미커밋). 변경 전 이미지 500이 연두에 보존됨
t2: TX2의 SELECT는 t2 시점에 '커밋된' 값을 Undo로 재구성해 즉시 반환
→ TX1의 미커밋 380은 보이지 않음 → 반환값 = 500

결과(오라클) = 500

[SQL Server] - 기본 RC(잠금 기반), RCSI 미사용

t1: TX1이 380으로 갱신하며 해당 행에 배타(X) 락 보유(미커밋)
t2: TX2의 SELECT는 공유(S) 락을 얻으려다 X 락과 비호환 → 대기(블로킹)
t3: TX1이 COMMIT → 380 확정 → X 락 해제 → TX2의 SELECT가 그제야 진행

→ 반환값 = 380
 결과(SQL Server) = 380

핵심 차이는 오라클은 읽기가 쓰기를 기다리지 않고(비블로킹) 커밋된 이전 값 500을 보고, 기본 SQL Server는 미커밋 X 락에 막혀 대기했다가 커밋 후 값 380을 본다는 점입니다. 따라서 (오라클, SQL Server) = (500, 380), 정답은 ④입니다.

RCSI를 켜면 SQL Server도 스냅샷으로 500을 읽어 두 값이 같아지지만, 발문의 (단, ... RCSI를 사용하지 않는다) 조건이 이 경로를 차단합니다.

선택지	판정	이유
①	×	두 값을 서로 뒤바꾼 것입니다. 비블로킹으로 500을 읽는 쪽이 오라클이고, 대기 후 380을 읽는 쪽이 SQL Server입니다
②	×	SQL Server를 오라클처럼 취급했습니다. 기본 RC(RCSI 미사용)에서 SELECT는 X 락에 막혀 대기하므로 500이 아니라 380입니다
③	×	오라클을 SQL Server처럼 취급했습니다. 오라클은 MVCC로 커밋된 이전 값 500을 즉시 읽으므로 380이 아닙니다
④	○	오라클은 MVCC로 미커밋 변경을 건너뛰고 500, 기본 SQL Server는 X 락에 블로킹됐다가 커밋 후 380을 읽습니다

▪ 이 문제의 핵심

1. 기본 격리수준은 둘 다 READ COMMITTED지만, 구현이 다릅니다 — 오라클은 MVCC, 기본 SQL Server는 잠금 기반입니다.
2. 오라클: 읽기가 쓰기를 막지 않음. 미커밋 변경은 언두로 되돌린 커밋 값(500)을 비블로킹으로 읽습니다.
3. 기본 SQL Server: 읽기가 미커밋 쓰기에 블로킹. SELECT가 공유 락을 얻으려다 대기하고, 커밋 후의 값(380)을 읽습니다.
4. RCSI(READ_COMMITTED_SNAPSHOT)를 켜면 SQL Server도 스냅샷으로 500을 읽어 오라클과 같아집니다 — 발문의 차단 조건이 이 예외를 막습니다.

▪ 한 줄 정리 같은 READ COMMITTED라도 오라클은 MVCC로 커밋된 이전 값 500을 비블로킹으로 읽고, RCSI를 끈 SQL Server는 X 락에 막혔다가 커밋 후 값 380을 읽습니다.